ObsidianDQ: A Local Hybrid Agentic Framework for Autonomous Data Quality Investigation and Remediation

Koushal V
Artificial Intelligence and Data Science
Rajalakshmi Engineering College Chennai, India
koushaljayanthi@gmail.com

Dr. Priyadharsini Ravisankar,
Professor
Artificial Intelligence and Data Science
Rajalakshmi Engineering College Chennai, India
priyadharsiniravisankar@rajalakshmi.edu.in

Jegadeeswaran D
Artificial Intelligence and Data Science
Rajalakshmi Engineering College Chennai, India
adjujeggi@gmail.com

Karthick S
Artificial Intelligence and Data Science
Rajalakshmi Engineering College Chennai,
h2karthi04@gmail.com

Abstract - Modern analytical pipelines depend on multi-stage Extraction, Load, and Transformation (ELT/ETL) workflows in which data quality problems can propagate across multiple downstream stages. Issues such as schema drift, negative values, missing primary keys, invalid dates, duplicate records, and incorrect join keys can remain undetected during ingestion and later affect reports, analytical models, and machine learning systems. Traditional data quality systems are effective at detecting predefined violations, but they generally provide alerts without automatically investigating the underlying cause or proposing a complete remediation workflow. At the same time, directly granting Large Language Model (LLM) agents access to databases introduces operational risks such as hallucinated schema information, invalid SQL generation, and unauthorized data modification.

This paper presents ObsidianDQ, a local hybrid agentic framework for autonomous data quality investigation and remediation. The framework separates reasoning from execution by restricting LLM-based agents to read-only diagnostic operations while assigning data scanning, lineage traversal, SQL transformation, partitioning, and verification to deterministic components. The system uses a stateful multi-agent workflow consisting of Investigation, Root-Cause, Planning, Triage, and Critic agents. Human-in-the-Loop approval is triggered for high-risk plans, low-confidence decisions, review-required actions, or exhausted critic iterations. SQL healing is performed using Abstract Syntax Tree parsing with sqlglot, while corrupted records are isolated through non-destructive quarantine operations. Evaluation using a 33-contract test suite and a controlled 20-incident benchmark achieved a 100.0% Controlled Incident Handling Success Rate and 100.0% Controlled Verification Consistency. The evaluation also demonstrates that the framework fails safely under external LLM rate limiting by routing uncertain cases to human review rather than executing unvalidated actions.

Keywords: Data Quality, Agentic AI, LangGraph, Data Observability, Data Lineage, SQL Self-Healing, Abstract Syntax Tree, Human-in-the-Loop, DuckDB.

I. INTRODUCTION

Modern analytical systems are built using multi-stage data pipelines that collect information from transactional systems, third-party sources, files, and other external systems. These inputs are transformed through staging tables, views, analytical models, and reporting layers. As the number of pipeline stages increases, a data quality problem introduced at an upstream stage can propagate to several downstream components.

Common examples include missing values, duplicate records, invalid categorical values, incorrect foreign keys, schema changes, invalid dates, and unexpected numerical values. These problems may not always cause an immediate pipeline failure. Instead, they can silently produce incorrect analytical results. The resulting errors can affect dashboards, reports, decision-making systems, and machine learning models.

Traditional data quality systems address this problem mainly through predefined assertions and monitoring rules. Tools such as Great Expectations and other data observability platforms can detect violations and generate alerts. However, detection does not necessarily provide an explanation of where the problem originated or how the affected transformation should be repaired. An engineer may still need to inspect the pipeline lineage, examine the affected records, identify the upstream cause, modify the transformation logic, isolate corrupted records, and verify the result.

Recent advances in Large Language Models have created opportunities for automating parts of data engineering and investigation. LLMs can reason over natural language descriptions, generate SQL, and interact with tools. However, allowing an LLM to directly execute database modifications is unsafe for production data systems. An LLM may hallucinate table or column names, generate syntactically invalid SQL, misunderstand schema relationships, or produce a destructive operation.

ObsidianDQ addresses this problem using a hybrid architecture in which reasoning and execution are explicitly separated. LLM agents are responsible for investigation, reasoning, planning, and triage. They operate through read-only diagnostic tools and cannot directly modify source data or database structures. Deterministic components perform data-quality checks, lineage traversal, SQL transformation, quarantine, and verification.

The main contributions of this work are:

1. A hybrid architecture that separates LLM reasoning from deterministic execution.
2. A lineage-based root-cause investigation mechanism for tracing downstream data quality failures to upstream sources.
3. An AST-based SQL healing mechanism that performs deterministic query transformation instead of unrestricted SQL generation.

4. A non-destructive remediation mechanism that preserves raw data and isolates corrupted records into quarantine partitions.
5. A stateful Human-in-the-Loop mechanism that prevents high-risk or low-confidence decisions from being executed automatically.
6. A controlled evaluation methodology that distinguishes deterministic incident handling from genuine LLM task success and reports verification failures truthfully.

II. Related Work

A. Traditional Data Quality and Observability

Traditional data quality systems generally use predefined expectations, statistical profiling, and validation rules to identify data quality violations. These systems are effective when the expected data characteristics are known in advance. However, conventional assertion-based systems primarily answer whether a rule has failed. They do not necessarily explain why the failure occurred or trace the failure through the dependency structure of a data pipeline. For example, a downstream fact table may contain invalid customer identifiers because an upstream customer file changed its column name. A basic assertion can identify the invalid identifiers, but an engineer still needs to investigate the upstream dependency.

ObsidianDQ extends deterministic data quality detection with lineage-based investigation and controlled remediation. The detection layer remains deterministic, while the agentic layer provides contextual investigation.

B. LLM-Based Code Generation and Text-to-SQL

Large Language Models have demonstrated strong capabilities in code generation and Text-to-SQL tasks. However, enterprise data environments contain schema-specific constraints, database dialect differences, foreign key relationships, and business-specific naming conventions.

An LLM-generated SQL query can therefore be syntactically valid while still being semantically incorrect. Hallucinated column names and incorrect joins are particularly problematic when the generated query is allowed to modify production data.

ObsidianDQ therefore does not rely on unrestricted LLM-generated SQL for remediation. SQL healing is performed through AST parsing and deterministic identifier rewriting using sqlglot. The LLM can identify and explain a potential problem, but the actual transformation is controlled by deterministic logic.

C. Agentic Workflow Systems and Human-in-the-Loop

Frameworks such as LangGraph and AutoGen support stateful multi-agent workflows in which specialized agents can perform different tasks and communicate through structured state.

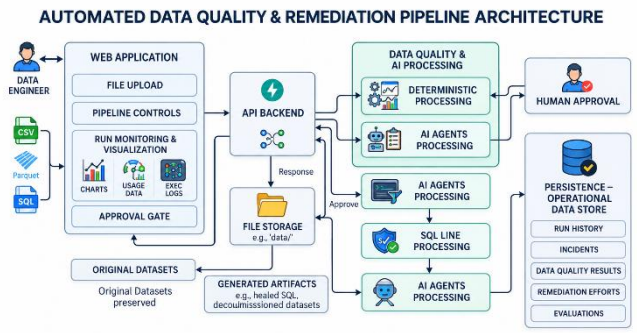
For data infrastructure, however, agentic autonomy requires explicit safety boundaries. An agent should not have unrestricted access to database mutation operations simply because it can generate an apparently correct plan.

ObsidianDQ introduces checkpointed Human-in-the-Loop approval, bounded critic iterations, read-only diagnostic tools, and deterministic execution components. These mechanisms allow agentic reasoning while retaining engineering control over high-risk actions.

III. System Architecture and Methodology

ObsidianDQ is organized into four major functional layers:

Figure 1. Overall Architecture of the ObsidianDQ Framework



1. Deterministic data quality detection
2. Lineage-based root-cause analysis
3. Stateful multi-agent investigation and planning
4. Deterministic remediation and verification

The central design principle is that the LLM reasons about the incident but does not directly execute destructive operations.

A. Deterministic Data Quality Detection

The first stage performs vectorized data quality checks over the ingested dataset. The benchmark pipeline uses stg_orders.parquet as the primary staging table and raw_customers.csv as an upstream dependency.

The implemented validation rules are:

- NOT_NULL
- NO_DUPLICATES
- PRICE_NON_NEGATIVE
- VALID_STATUS
- UNIQUE_ORDER_ID
- ORDER_DATE_RANGE

Each detected violation is assigned a severity. A composite pipeline health score is then calculated as:

H = max(0, min(100, 100 - (15N_HIGH + 5N_MED + 2N_LOW)))

where:

- H represents the overall pipeline health score.
- N_HIGH represents the number of high-severity issues.
- N_MED represents the number of medium-severity issues.
- N_LOW represents the number of low-severity issues.

The deterministic detector provides the initial incident state that is passed to the investigation subsystem.

B. Lineage-Based Root-Cause Analysis

The pipeline is represented as a Directed Acyclic Graph:

$$G = (V, E)$$

where V represents tables, views, or files and E represents dependency relationships between pipeline components.

For a node v, the upstream ancestors and downstream descendants are defined as:

$$Ancestors(v) = \{u \; in \; V \mid u \; reaches \; v\}$$

$$Descendants(v) = \{w \; in \; V \mid v \; reaches \; w\}$$

The number of descendants provides an estimate of the downstream blast radius of an affected node:

$$BlastRadius(v) = |Descendants(v)|$$

This information helps determine whether a failure is isolated to a single artifact or can affect multiple downstream analytical components.
Candidate root causes are identified by examining upstream nodes in the transitive failure subgraph. Nodes with zero incoming dependencies within the affected subgraph are treated as candidate origin points.
This lineage information is supplied to the agentic subsystem as evidence rather than allowing the LLM to independently infer the pipeline topology.

C. Agentic Diagnostic and Triage Subsystem

After deterministic detection and lineage analysis, execution transitions to the multi-agent subsystem.
Figure 2. Stateful Multi-Agent Investigation and Remediation Workflow

1. Investigation Agent
The Investigation Agent performs read-only diagnostic operations using four tools:

- **get_sample_rows(column, condition):** Retrieves a limited number of offending records.
- **get_column_stats(column):** Computes null ratio, cardinality, and data type information.
- **get_lineage_context(stage):** Retrieves upstream lineage and downstream blast radius.
- **search_similar_incidents(column, rule):** Searches historical incidents stored in DuckDB for similar cases.

The Investigation Agent only collects evidence and does not modify the dataset.

2. Root-Cause Agent
The Root-Cause Agent combines investigation evidence with lineage information to identify the most likely failure origin.
The output contains:

- Failure origin stage
- Technical reasoning
- Supporting evidence
- Causality explanation
- upstream_causality_proven status

The objective is to determine whether the evidence supports an upstream cause.

3. Planning Agent
The Planning Agent creates a structured remediation plan using the mandatory sequence:

$$INVESTIGATE \rightarrow SQL\_HEAL \rightarrow QUARANTINE \rightarrow VERIFY$$

The plan is assigned one of three risk levels:

- LOW
- MEDIUM
- HIGH

The plan is non-executing and must pass governance conditions before execution.

4. Triage Agent
The Triage Agent generates a containment proposal for each identified issue.
Possible actions are:

- AUTO_QUARANTINE
- FLAG_FOR_REVIEW
- IGNORE_TRANSIENT
- ESCALATE_UPSTREAM

Each proposal contains a confidence value: $\mathbf{C \in [0.0, 1.0]}$
Low-confidence decisions are not automatically executed.

5. Critic Agent
The Critic Agent reviews the generated remediation plan and triage proposals.
It returns:

- **APPROVED**
- **REVISION_REQUIRED**

If **REVISION_REQUIRED** is returned, execution returns to the Investigation Agent. The critic loop allows a maximum of one retry. If the second attempt fails, the system requires human review and this bounded loop prevents indefinite agentic execution.

IV. HUMAN-IN-THE-LOOP GOVERNANCE

Human approval is required when the proposed remediation exceeds the configured autonomous execution safety threshold. The review condition is defined as:

$$\boldsymbol{RequiresReview} = (\boldsymbol{R\_plan} = \boldsymbol{HIGH})\ \boldsymbol{OR}\ (\exists\ \boldsymbol{proposal}:\ \boldsymbol{action} = \boldsymbol{FLAG_{FOR_{REVIEW}}} \vee \boldsymbol{confidence} < \mathbf{0.70})\boldsymbol{OR}\ (\boldsymbol{Critic} = \boldsymbol{REVISION\_EXHAUSTED})$$

When RequiresReview is true, the LangGraph workflow is paused at the human_review_queue node. The workflow state is checkpointed using the LangGraph MemorySaver, and the pipeline status is set to:
WAITING_FOR_HUMAN_APPROVAL
An authenticated API request is required to resume the paused workflow.
If the engineer approves the proposed remediation, execution proceeds to the SQL healing and quarantine stages. If the engineer rejects the proposal, the workflow terminates with the status:
APPROVAL_REJECTED
No remediation operation is performed on the raw source data or affected database tables when the proposal is rejected. This governance mechanism establishes an explicit boundary between autonomous diagnostic reasoning and human authorization, preventing high-risk or low-confidence remediation actions from being executed without approval.

Figure 3. Multi-Agent Recommendation and Human Approval Gate

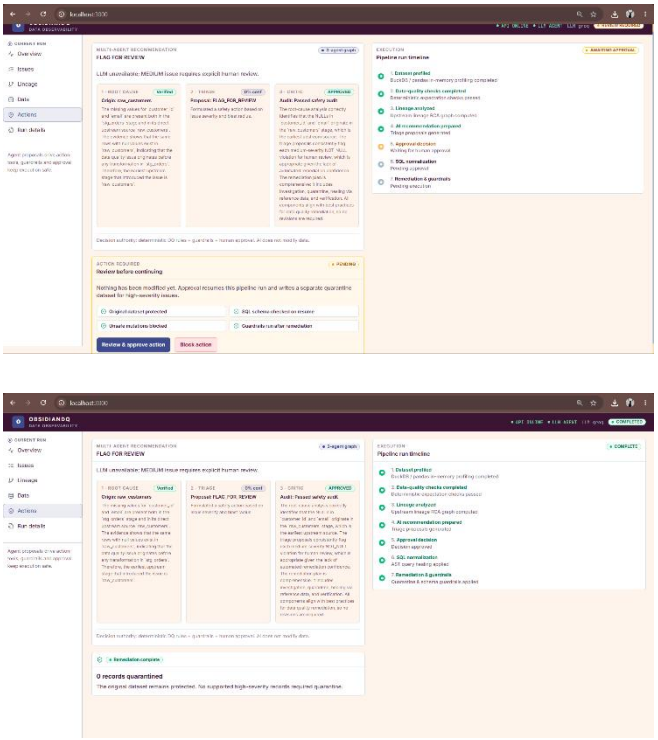


Figure 3 shows the human approval interface generated from the agentic investigation and triage results. The interface exposes the proposed action, confidence information, and approval decision to the engineer. High-risk or low-confidence proposals are therefore presented for explicit human authorization before deterministic remediation is executed.

V. AST-GROUNDED SQL HEALING

SQL transformation failures can occur when upstream schemas change or when column aliases become outdated.
ObsidianDQ uses sqlglot to parse SQL into an Abstract Syntax Tree. The repair process consists of the following steps:

1. Parse the original SQL query into an AST.
2. Identify referenced tables and compare them with the available physical data files.
3. Inspect column references, particularly identifiers used in JOIN conditions.
4. Determine whether the referenced identifier exists in the corresponding upstream schema.
5. Apply a deterministic identifier replacement when a valid mapping is established.
6. Generate the repaired SQL query from the modified AST.

For example, the benchmark transformation contains a join condition: ON o.customer_id = c.cust_id
If cust_id does not exist in raw_customers.csv but customer_id is the valid corresponding identifier, the AST healer modifies the relevant expression node.
The repaired query is then written to the healed-query directory as a separate artifact.

The important safety property is that the LLM does not directly modify the SQL string and execute the result. The transformation is controlled by deterministic AST operations.

Figure 4. AST-Grounded SQL Healer Diff Viewer

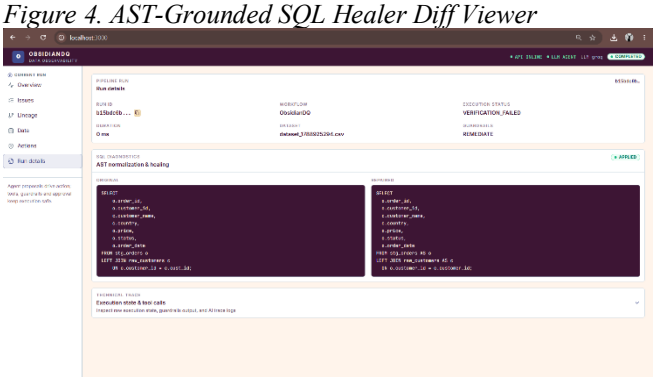


Figure 4 presents the frontend representation of the SQL healing operation. The interface allows the engineer to inspect the original and transformed SQL representations and review the deterministic change produced by the AST-based transformation. This provides visibility into the proposed SQL correction without allowing the LLM to directly modify or execute the query.

VI. NON-DESTRUCTIVE REMEDIATION

ObsidianDQ does not modify the original raw data during remediation.
Records identified for containment are written to a separate quarantine partition:
data/quarantine/stg_orders_quarantine.parquet
Clean records are exported to a derived target:
data/cleaned/stg_orders_cleaned.parquet
This separation provides three important properties:

1. The original source remains available for audit and recovery.
2. Corrupted records are isolated instead of being silently deleted.
3. The cleaned dataset becomes a derived artifact that can be independently verified.

The remediation process therefore follows the principle of preserving evidence before applying corrective operations.

VII. VERIFICATION

After remediation, the complete deterministic data quality detection suite is executed again against the cleaned artifact. If no violations are detected, verification_passed = True; otherwise, the workflow transitions to VERIFICATION_FAILED. The system reports successful remediation only when post-remediation verification passes, since successful execution of a remediation operation does not necessarily mean that the underlying data quality issue has been resolved. Thus, the verification stage serves as the final validation gate before reporting successful remediation.

VIII. IMPLEMENTATION

ObsidianDQ is implemented as a full-stack system containing backend services, an embedded analytical database, an agentic workflow, deterministic data processing components, and a web interface.

A. Backend

The backend uses FastAPI to expose REST endpoints for dataset ingestion, pipeline execution, and human approval. The backend coordinates:

- Dataset ingestion
- Pipeline execution
- Agent workflow execution
- Human approval
- Verification
- Incident persistence

B. Persistence

DuckDB is used as the embedded analytical database. The database stores information related to:

- Pipeline runs
- Detected incidents
- AST evidence
- Evaluation records
- Historical incident information

Using DuckDB provides a local analytical persistence layer without requiring a separate database server for the prototype.

C. Agent Workflow

LangGraph manages the stateful multi-agent workflow. The workflow coordinates the Investigation, Root-Cause, Planning, Triage, Critic, Human Review, Remediation, and Verification stages. The stateful workflow also enables the system to pause for human approval and resume from the preserved execution state.

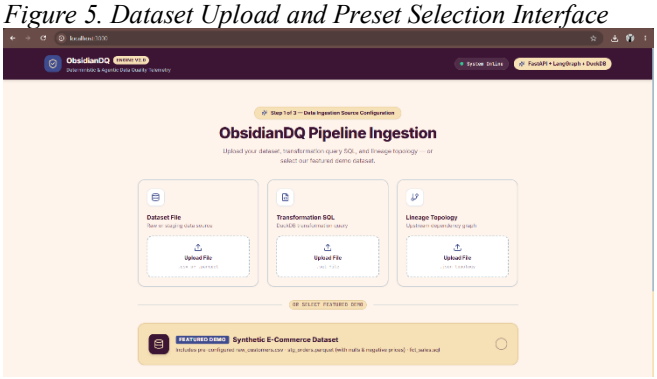
D. Web Console

The frontend is implemented using Next.js 14 and React. The web console provides:

- Interactive pipeline views
- DAG visualization using @xyflow/react
- Run metrics
- SQL AST difference inspection
- Human approval interface

The interface allows an engineer to inspect the reasoning and proposed remediation before approving high-risk actions.

Figure 5. Dataset Upload and Preset Selection Interface



The web console provides the interface through which an engineer can upload a dataset, select the applicable validation configuration, execute a pipeline run, inspect detected incidents, review lineage information, and approve or reject remediation proposals.

Figure 6. Executive Run Console Overview

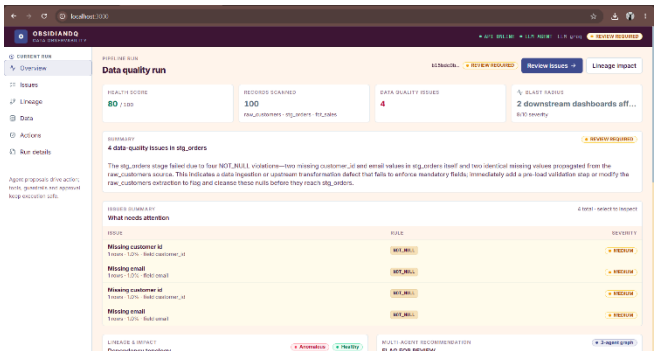


Figure 6 presents the run-level overview, including pipeline status, detected incidents, health information, and execution results. The interface provides a consolidated view of the current pipeline run before detailed investigation and remediation.

Figure 7. Interactive Pipeline Lineage DAG

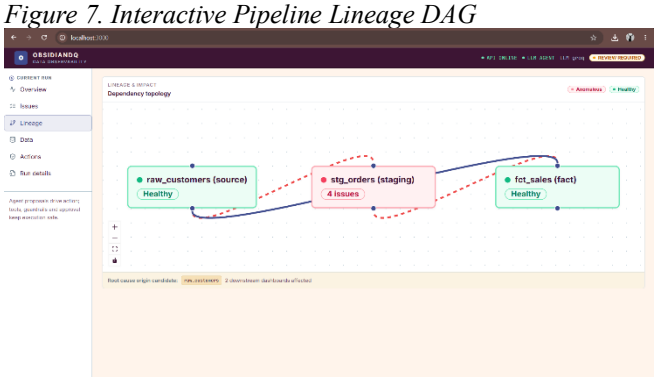


Figure 7 provides the visual representation of the pipeline dependency graph used during root-cause investigation. The DAG allows the engineer to inspect upstream dependencies and downstream descendants associated with an incident. This visualization complements the deterministic lineage analysis used to estimate the potential downstream blast radius.

IX. EXPERIMENTAL EVALUATION

The evaluation consists of two levels:

1. Automated contract testing
2. Controlled incident benchmarking

The system contains a 33-contract automated test suite and a controlled benchmark containing 20 injected incidents.

A. Evaluation Grounding Rules

The evaluation separates deterministic correctness from LLM reasoning quality.

First, passing automated Pytest contracts demonstrates that the implementation satisfies the defined architectural contracts. It does not by itself demonstrate that an LLM has successfully reasoned about an incident.

Second, deterministic issue isolation in offline fallback mode is counted as controlled incident handling. Agent Task Success is not reported for cases where genuine LLM reasoning was not measured.

Third, verification is evaluated according to the actual state of the cleaned dataset. If the dataset remains corrupted and the system correctly reports VERIFICATION_FAILED, this is considered correct verification behavior.

Fourth, live LLM metrics are calculated only from genuine non-fallback traces.
These rules prevent deterministic fallback behavior from being incorrectly presented as successful autonomous agent reasoning.

B. Controlled Incident Benchmark

The controlled benchmark contains 20 incidents representing different data quality failures.
Across the benchmark, ObsidianDQ achieved:

- **Controlled Incident Handling Success Rate: 100.0%**
- **Successful Cases: 20/20**

All injected anomalies were isolated and matched against the expected oracle keys.
Post-remediation validation achieved:

- **Controlled Verification Consistency: 100.0%**

This result includes the case in which a date corruption could not be resolved. Instead of reporting a false success, the system correctly halted at VERIFICATION_FAILED.

C. Live LLM Evaluation

During live Groq evaluation, external API rate limiting caused 82 call failures across multi-turn interactions.
Rather than allowing the failure to result in uncontrolled execution, the system transitioned affected triage decisions to: FLAG_FOR_REVIEW
with: confidence = 0.0
The workflow subsequently routed the case to the Human Approval Queue. This behavior demonstrates the fail-safe property of the architecture under external LLM availability problems.

X. Discussion

The experimental results support the central architectural principle of ObsidianDQ: critical data infrastructure should not depend on unrestricted LLM autonomy. The LLM subsystem provides reasoning capabilities for investigation, root-cause analysis, planning, and triage. However, it does not receive direct authority to modify source data or database structures. Deterministic components retain control over the operations where correctness and safety are critical. These include data validation, lineage traversal, AST transformation, partitioning, and verification. This separation also provides resilience when the external LLM service is unavailable. Instead of treating an LLM failure as permission to continue with an uncertain action, the system falls back to controlled behavior and routes uncertain cases for human review. The approach therefore combines the flexibility of agentic reasoning with the predictability of deterministic execution.

XI. LIMITATIONS

The current implementation has several limitations.

A. External LLM Availability: Live multi-agent investigation depends on external LLM API availability. Free-tier or externally imposed quotas can result in HTTP 429 rate-limit errors during multi-turn tool interactions.

B. Critic Fallback: If the Critic Agent's LLM call fails, the current implementation can default to approving the run. Although downstream controls can still detect high-risk proposals, the critic itself should ideally fail closed. This is an identified safety limitation and should be addressed before treating the system as production-ready.

C. In-Memory Checkpointing: The current LangGraph workflow uses MemorySaver for checkpointing. Because the state is held in process memory, restarting the backend can remove paused approval states. A persistent checkpointing mechanism such as PostgreSQL would be required for reliable deployment across service restarts.

D. Schema-Coupled Rules: Several validation rules are currently designed around the benchmark e-commerce schema. Rules such as PRICE_NON_NEGATIVE and ORDER_DATE_RANGE are therefore not fully domain independent. Supporting arbitrary data domains would require a configuration-driven rule specification or a declarative rule DSL.

XII. Conclusion

This paper presented ObsidianDQ, a local hybrid agentic framework for data quality investigation and controlled remediation.
The framework combines deterministic data quality detection, lineage-based root-cause analysis, multi-agent investigation, AST-grounded SQL healing, non-destructive quarantine, Human-in-the-Loop governance, and post-remediation verification.
The central design principle is the separation of reasoning from execution. LLM agents are restricted to read-only diagnostic operations and structured proposals, while deterministic components retain authority over data scanning, lineage traversal, SQL transformation, partitioning, and verification.
Evaluation using a 33-contract test suite and a controlled 20-incident benchmark achieved a 100.0% Controlled Incident Handling Success Rate and 100.0% Controlled Verification Consistency. The system also demonstrated controlled degradation under external LLM rate limiting by routing affected cases to human review.
Future work will focus on four main areas. First, the in-memory checkpoint mechanism can be replaced with distributed persistent storage such as PostgreSQL. Second, the current schema-specific validation rules can be extended into a declarative YAML-based rule specification system. Third, the safety critic can be modified to fail closed whenever its validation step cannot be completed. Finally, a larger open benchmark for semantic root-cause analysis can be developed to evaluate agent reasoning beyond the current controlled incidents.

References

[1] B. Moses, L. Gavish, and D. Vorwerck, *Data Quality Fundamentals: A Practitioner's Guide to Building Trustworthy Data Pipelines*. Sebastopol, CA, USA: O'Reilly Media, 2022.

[2] C. Batini, C. Cappiello, C. Francalanci, and A. Maurino, "Methodologies for data quality assessment and improvement," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1-52, 2009.

[3] T. Mao et al., "SQLGlot: An extensible SQL parser and transpiler," 2024. [Online]. Available: https://github.com/tobymao/sqlglot.

[4] M. Raasveldt and H. Mühleisen, "DuckDB: An embeddable analytical database," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2019, pp. 1981-1984.

[5] C. Monperrus, "A survey on automatic software repair," *ACM Computing Surveys*, vol. 51, no. 1, pp. 1-37, 2018.

[6] LangChain AI, "LangGraph: Building stateful, multi-actor applications with LLMs," 2024. [Online]. Available: https://github.com/langchain-ai/langgraph.

[7] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing incrementally for constrained auto-regressive decoding from language models," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2021, pp. 9895-9901.

[8] D. Pourreza and H. Rafiei, "DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[9] Great Expectations Consortium, "Standardizing declarative data assertions for modern pipelines," Superconductive, 2023. [Online]. Available: https://greatexpectations.io/.

[10] Apache Airflow Consortium, "Data lineage tracking in directed acyclic graphs," Apache Software Foundation, 2024.

[11] S. Tiangolo, "FastAPI: High performance modern web framework," 2024. [Online]. Available: https://fastapi.tiangolo.com/.

[12] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv preprint arXiv:2308.08155, 2023.